



Assignment 3

Document Retrieval-Augmented Generation System

From Static Pipelines to Deployed Intelligent Document Search

Natural Language Processing
Academic Year 2025-2026

March 1, 2026

Assignment Weight: 5% of Final Grade

Deadline: March 11th, 2026 — via MiAulario

Submission Format: GitHub repository link + technical report (max 3 pages)

Prerequisites: Assignments 1 & 2, Docker basics, REST APIs

Contents

1	Introduction and Objectives	4
1.1	The Problem We Are Solving	4
1.2	Learning Objectives	4
2	System Architecture	5
2.1	Required Components	5
2.2	High-Level Architecture	5
2.3	Request Flows	5
3	Mandatory Requirements	6
3.1	R1: Containerized Infrastructure	6
3.1.1	LLM Service: llama.cpp or vLLM	7
3.1.2	MinIO Object Store	8
3.2	R2: Document Parsing	8
3.3	R3: Chunking Strategy	9
3.4	R4: Retrieval Method	9
3.5	R5: Flask REST API	10
4	Optional Extension: Agentic Pipeline	12
5	Bonus: Frontend Interface	13
6	Evaluation and Retrieval Metrics	13
6.1	Measuring Retrieval Quality	13
6.1.1	Creating the Evaluation Dataset	13
6.1.2	Metrics to Report	14
6.2	Qualitative Analysis	14
7	Technical Guidance	15
7.1	Calling the LLM from Flask	15
7.2	ChromaDB as Vector Store	16
7.3	BM25 Retrieval Alternative	16
7.4	Environment Variables and Service Discovery	17

8 Deliverables and Submission	17
8.1 What to Submit	17
8.2 README Requirements	18
8.3 Report Requirements (Max 3 Pages)	18
9 Evaluation Criteria	19
10 Frequently Asked Questions	19
11 Conclusion	20

1 Introduction and Objectives

Assignments 1 and 2 focused on model training: classical baselines, neural sequence models, and rigorous evaluation methodology. You now understand the trade-offs between classical and deep learning approaches and have hands-on experience with embeddings and attention. This assignment takes a different direction: instead of training models, you will **design, build, and deploy a complete document intelligence system**.

The real world of applied NLP is dominated not by training new models, but by **engineering systems** that combine existing components intelligently. Retrieval-Augmented Generation (RAG) is the production standard for document question answering, enterprise knowledge management, and intelligent search. Building one end-to-end — from document ingestion to grounded generation — is the single most practical skill you will develop in this course.

1.1 The Problem We Are Solving

Consider a company with thousands of internal PDF and Word documents: policies, contracts, technical manuals, reports. When an employee asks a question, they cannot read everything. A RAG system solves this by:

1. **Ingesting** documents (PDF, DOCX) and extracting their text content.
2. **Chunking** the text into retrievable units suitable for semantic search.
3. **Indexing** those chunks using an embedding or keyword-based method.
4. **Retrieving** the most relevant chunks for any user query at inference time.
5. **Generating** a grounded, cited answer using a local language model.

Your system will run **entirely locally** — no external APIs, no cloud costs — using a quantized LLM served via Docker.

1.2 Learning Objectives

By completing this assignment, you will:

- Design and deploy a multi-container application using Docker and Docker Compose.
- Build a document ingestion pipeline supporting PDF and DOCX formats.
- Implement at least one retrieval strategy from a spectrum of options (sparse to dense).
- Integrate a quantized local LLM via llama.cpp or vLLM for grounded generation.
- Use MinIO as a persistent object store for raw document management.
- Expose a well-designed REST API (Flask) for document upload and question answering.
- Evaluate retrieval quality using standard IR metrics on a hand-annotated dataset.

Important

This assignment is intentionally open-ended. The minimum requirements define the floor of what is needed for a passing grade. You have significant freedom in your retrieval method, chunking strategy, pipeline architecture, and optional extensions. A clean, well-documented minimum system beats a complex broken one. Prioritize correctness and reproducibility over ambition.

2 System Architecture

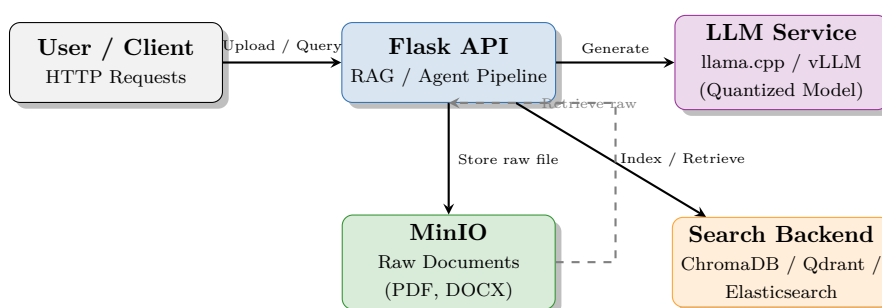
2.1 Required Components

Your final system **must** contain these four components, each running in its own Docker container, orchestrated by a single `docker-compose.yml` file:

Component	Purpose	Technology
LLM Service	Serve a quantized language model for answer generation	llama.cpp (llama-server) or vLLM
Object Store	Persist raw uploaded documents	MinIO
RAG / Agent API	Document ingestion, retrieval, generation	Flask (Python)
Search Backend	Index and query embeddings or sparse vectors	ChromaDB, Qdrant, Elasticsearch, or SQLite

Table 1: Mandatory system components

2.2 High-Level Architecture



The Flask API is the central orchestration layer. It receives user requests, manages document storage in MinIO, coordinates retrieval from the search backend, and calls the LLM service to produce final answers.

2.3 Request Flows

Document Upload Flow:

1. Client sends `POST /documents` with a file (PDF or DOCX).
2. Flask stores the raw file in **MinIO**.
3. Flask parses and extracts clean text from the document.
4. Flask chunks the text using the chosen strategy.
5. Flask embeds or indexes the chunks in the **search backend**.
6. Flask returns a document ID and a brief ingestion summary.

Query Flow:

1. Client sends `POST /query` with a question string.
2. Flask retrieves the top- k most relevant chunks from the search backend.
3. Flask assembles a prompt: system instruction + retrieved context + user question.
4. Flask calls the **LLM service** with the assembled prompt.
5. Flask returns the generated answer plus the source chunks used.

3 Mandatory Requirements

3.1 R1: Containerized Infrastructure

Every component must run inside a Docker container. You must provide:

- A `Dockerfile` for the Flask application (and any custom service wrapper).
- A `docker-compose.yml` that starts the entire system with `docker-compose up`.
- No manual setup steps beyond cloning the repository, downloading the model (see below), and running Docker Compose.

```
1  assignment3/
2  |-- docker-compose.yml
3  |-- flask_app/
4  |   |-- Dockerfile
5  |   |-- app.py
6  |   |-- requirements.txt
7  |   |-- ingestion/
8  |   |   |-- parser.py           # PDF + DOCX parsing
9  |   |   |-- chunker.py         # Chunking strategy
10 |   |-- retrieval/
11 |   |   |-- retriever.py        # Chosen retrieval method
12 |   |-- pipeline/
13 |   |   |-- rag.py              # RAG or agent orchestration
14 |-- models/
15 |   |-- README.md               # Download instructions for model weights
16 |-- report/
17 |   |-- report.pdf
```

Listing 1: Expected repository structure

3.1.1 LLM Service: llama.cpp or vLLM

Choose **one** of the following. Both expose an OpenAI-compatible REST API that your Flask application will call.

Option A: llama.cpp (llama-server) Recommended for machines without a GPU or with limited VRAM. Supports CPU inference with GGUF quantized models.

```
1  services:
2    llm:
3      image: ghcr.io/ggerganov/llama.cpp:server
4      ports:
5        - "8080:8080"
6      volumes:
7        - ./models:/models
8      command: >
9        -m /models/mistral-7b-instruct-v0.2.Q4_K_M.gguf
10       --host 0.0.0.0 --port 8080
11       --ctx-size 4096 --n-predict 512
```

Listing 2: llama.cpp service in docker-compose.yml

Option B: vLLM Recommended for machines with an NVIDIA GPU. Supports GPTQ and AWQ quantized models.

```
1  services:
2    llm:
3      image: vllm/vllm-openai:latest
4      ports:
5        - "8000:8000"
6      volumes:
7        - ~/.cache/huggingface:/root/.cache/huggingface
8      deploy:
9        resources:
10         reservations:
11           devices:
12             - driver: nvidia
13               count: 1
14               capabilities: [gpu]
15      command: >
16        --model TheBloke/Mistral-7B-Instruct-v0.2-GPTQ
17        --quantization gptq
```

Listing 3: vLLM service in docker-compose.yml

Suggested models that balance quality with local hardware constraints:

- **Mistral 7B Instruct v0.2** (GGUF Q4_K_M) — strong instruction following, runs on CPU at reasonable speed.
- **Llama 3.1 8B Instruct** (GGUF Q4_K_M) — excellent multilingual capability.
- **Qwen2.5 3B Instruct** (GGUF Q5_K_M) — compact and fast, ideal for low-memory machines.
- Any other publicly available GGUF or quantized model you prefer.

3.1.2 MinIO Object Store

```

1  services:
2    minio:
3      image: minio/minio:latest
4      ports:
5        - "9000:9000"
6        - "9001:9001"      # Web console (optional, useful for debugging)
7      environment:
8        MINIO_ROOT_USER: minioadmin
9        MINIO_ROOT_PASSWORD: minioadmin
10     volumes:
11       - minio_data:/data
12     command: server /data --console-address ":9001"
13
14  volumes:
15    minio_data:

```

Listing 4: MinIO service in docker-compose.yml

Your Flask application must interact with MinIO via the official minio Python client to upload, download, and list documents:

```

1  from minio import Minio
2  import io
3
4  minio_client = Minio(
5      "minio:9000",
6      access_key="minioadmin",
7      secret_key="minioadmin",
8      secure=False
9  )
10  BUCKET = "documents"
11
12  def upload_to_minio(file_bytes: bytes, filename: str, content_type: str):
13      if not minio_client.bucket_exists(BUCKET):
14          minio_client.make_bucket(BUCKET)
15      minio_client.put_object(
16          BUCKET, filename,
17          io.BytesIO(file_bytes), length=len(file_bytes),
18          content_type=content_type
19      )

```

Listing 5: Minimal MinIO client usage in Flask

3.2 R2: Document Parsing

Your ingestion pipeline must support at least **PDF** and **DOCX** files. The parsing step must extract clean plain text, including basic table and list content where possible.

```

1  import pdfplumber
2  from docx import Document
3  import io
4
5  def parse_pdf(file_bytes: bytes) -> str:
6      text_parts = []
7      with pdfplumber.open(io.BytesIO(file_bytes)) as pdf:
8          for page in pdf.pages:

```



```

9         text = page.extract_text()
10        if text:
11            text_parts.append(text)
12    return "\n\n".join(text_parts)
13
14    def parse_docx(file_bytes: bytes) -> str:
15        doc = Document(io.BytesIO(file_bytes))
16        paragraphs = [p.text for p in doc.paragraphs if p.text.strip()]
17        return "\n\n".join(paragraphs)
18
19    def parse_document(file_bytes: bytes, filename: str) -> str:
20        ext = filename.rsplit(".", 1)[-1].lower()
21        if ext == "pdf":
22            return parse_pdf(file_bytes)
23        elif ext in ("docx", "doc"):
24            return parse_docx(file_bytes)
25        else:
26            raise ValueError(f"Unsupported file type: {ext}")

```

Listing 6: Parser supporting PDF and DOCX

Common Pitfall

Scanned PDFs (image-only) will return empty text with pdfplumber. You are not required to support OCR, but you must detect this case and return a clear error message to the user rather than silently indexing an empty document.

3.3 R3: Chunking Strategy

Implement at least **one** of the following chunking strategies. More sophisticated strategies will be rewarded in the evaluation.

Strategy	Description
Fixed-Size	Sliding window of N tokens with O tokens of overlap. Simple, fast, production-standard baseline.
Recursive Splitting	Split by paragraph separator, then sentence, then word, until chunks fit within the size limit. Preserves author-intended boundaries.
Semantic Chunking	Embed each sentence; split where cosine similarity between consecutive sentences drops below a threshold θ .
Parent-Child	Index small child chunks for retrieval precision; return larger parent chunks to the LLM for fuller context.
Agentic Propositions	Use the LLM itself to decompose text into atomic, self-contained factual statements.

Table 2: Available chunking strategies (implement at least one)

3.4 R4: Retrieval Method

Implement at least **one** of the following retrieval strategies. You have full freedom. Methods are ordered approximately from simplest to most sophisticated.

Method	Description	Typical Backend
BM25 / TF-IDF	Sparse keyword matching. Fast, no GPU required. Solid baseline.	Elasticsearch, rank_bm25
Word2Vec / GloVe	Average static word vectors; cosine similarity for retrieval.	NumPy + pickle
Bi-Encoder (Dense)	Sentence transformer embeddings + approximate nearest neighbour search.	ChromaDB, Qdrant
Cross-Encoder Reranker	Rerank top- k dense results with a cross-encoder for higher precision.	sentence-transformers
Hybrid (BM25 + Dense)	Fuse sparse and dense rankings using Reciprocal Rank Fusion (RRF).	Elasticsearch + Qdrant
HyDE	Embed a hypothetical answer generated by the LLM instead of the raw query.	Any vector store

Table 3: Available retrieval methods (implement at least one)

Pro Tip

If you are unsure where to start, the recommended path is: **Fixed-Size Chunking + Bi-Encoder Retrieval (all-MiniLM-L6-v2) + ChromaDB**. This combination is well-documented, runs on CPU, and produces solid results with minimal engineering overhead. More sophisticated combinations are rewarded.

3.5 R5: Flask REST API

Your Flask application must expose the following endpoints:

Method	Endpoint	Description
POST	/documents	Upload a PDF or DOCX file. Returns document ID.
GET	/documents	List all documents (ID, filename, upload date).
DELETE	/documents/{id}	Delete a document and its indexed chunks.
POST	/query	Submit a question. Returns answer and source chunks.
GET	/health	Health check confirming all services are reachable.

Table 4: Mandatory REST API endpoints

```

1 from flask import Flask, request, jsonify
2 import uuid
3
4 app = Flask(__name__)
5

```

```

6 @app.route("/documents", methods=["POST"])
7 def upload_document():
8     if "file" not in request.files:
9         return jsonify({"error": "No file provided"}), 400
10    file = request.files["file"]
11    doc_id = str(uuid.uuid4())
12    # 1. Upload raw file to MinIO
13    # 2. Parse document (PDF or DOCX)
14    # 3. Chunk extracted text
15    # 4. Index chunks in the search backend
16    return jsonify({"doc_id": doc_id, "filename": file.filename}), 201
17
18 @app.route("/documents", methods=["GET"])
19 def list_documents():
20     return jsonify({"documents": []}), 200
21
22 @app.route("/documents/<doc_id>", methods=["DELETE"])
23 def delete_document(doc_id):
24     # Remove from MinIO and from vector / search store
25     return jsonify({"deleted": doc_id}), 200
26
27 @app.route("/query", methods=["POST"])
28 def query():
29     data = request.get_json()
30     question = data.get("question", "")
31     if not question:
32         return jsonify({"error": "No question provided"}), 400
33     # 1. Retrieve relevant chunks
34     # 2. Assemble prompt
35     # 3. Call LLM service
36     # 4. Return answer + sources
37     return jsonify({"answer": "", "sources": []}), 200
38
39 @app.route("/health", methods=["GET"])
40 def health():
41     return jsonify({"status": "ok"}), 200
42
43 if __name__ == "__main__":
44     app.run(host="0.0.0.0", port=5000, debug=False)

```

Listing 7: Minimal Flask API skeleton

The /query endpoint response must include at minimum:

```

1  {
2    "question": "What is the company remote work policy?",
3    "answer": "Employees may work remotely up to 3 days per week...",
4    "sources": [
5      {
6        "doc_id": "abc-123",
7        "filename": "hr_policy_2024.pdf",
8        "chunk_text": "Section 4.2: Remote work is permitted...",
9        "score": 0.87
10     }
11   ]
12 }

```

Listing 8: Expected /query JSON response

4 Optional Extension: Agentic Pipeline

This section is optional. If you implement it, it replaces the simple Retrieve → Generate pipeline and qualifies for bonus marks.

Instead of a fixed retrieval loop, an agent wraps the LLM in a decision cycle. At each step it can choose among available tools:

1. `search_documents(query)` — retrieve relevant chunks from the search backend.
2. `summarize(text)` — ask the LLM to condense a retrieved passage.
3. `final_answer(text)` — produce the answer and terminate the loop.

```

1  SYSTEM_PROMPT = """You are a document assistant with access to:
2  - search(query: str) -> list of relevant text chunks
3  - summarize(text: str) -> concise summary
4
5  Reason step by step using Thought / Action / Observation format.
6  When done, output: Final Answer: <your answer>"""
7
8  TOOLS = {"search": search_documents, "summarize": summarize_text}
9
10 def run_agent(question: str, max_steps: int = 5) -> str:
11     messages = [
12         {"role": "system", "content": SYSTEM_PROMPT},
13         {"role": "user", "content": question}
14     ]
15     for _ in range(max_steps):
16         response = call_llm(messages)
17         if "Final Answer:" in response:
18             return response.split("Final Answer:")[1].strip()
19         tool_name, tool_args = parse_action(response)
20         observation = TOOLS[tool_name](**tool_args)
21         messages.append({"role": "assistant", "content": response})
22         messages.append({"role": "user",
23                         "content": f"Observation: {observation}"})
24     return "Could not complete query within the step limit."

```

Listing 9: Minimal ReAct-style agent loop

5 Bonus: Frontend Interface

Bonus

A frontend interface is a **bonus task**. It is not required for a passing grade.

A minimal frontend should allow:

- Uploading PDF and DOCX documents via a file picker.
- Listing and deleting uploaded documents.
- Submitting a free-text query and displaying the generated answer plus source chunks.

Acceptable implementations:

- A static HTML + JavaScript page served by Flask or a lightweight Nginx container.
- A **Streamlit** or **Gradio** application running as an additional container.
- A React or Vue.js single-page application in its own container.

The frontend must be containerized and included in the `docker-compose.yml`. A rough but functional interface far outweighs a polished one requiring manual setup.

6 Evaluation and Retrieval Metrics

6.1 Measuring Retrieval Quality

You are required to **measure and report the retrieval quality** of your system. This involves building a small hand-annotated evaluation dataset and computing standard information retrieval metrics over it.

6.1.1 Creating the Evaluation Dataset

1. **Select 3–5 documents** from your document collection.
2. **Write 10–20 questions** that have clear, locatable answers in those documents.
3. **Annotate** which chunks (or text passages) are relevant to each question.
4. Store the dataset as `eval_dataset.json` with the structure shown below.

```
1  [  
2    {  
3      "question": "What is the annual remote work stipend?",  
4      "relevant_doc_id": "abc-123",  
5      "relevant_passage": "Employees receive a $500 remote work stipend...",  
6      "expected_answer": "$500 per year, submitted via Concur by Dec 31st"  
7    }  
8  ]
```

Listing 10: eval_dataset.json structure

6.1.2 Metrics to Report

Run your retrieval pipeline on each question with $k \in \{1, 3, 5\}$ and compute:

Metric	Definition
Hit Rate @ k	Fraction of questions for which at least one relevant chunk appears in the top k results.
Mean Reciprocal Rank (MRR)	$\text{MRR} = \frac{1}{ Q } \sum_{i=1}^{ Q } \frac{1}{\text{rank}_i}$ where rank_i is the position of the first relevant chunk. If no relevant chunk is found, the contribution is 0.
Precision @ k	Average fraction of the top- k results that are relevant.

Table 5: Required retrieval metrics

```

1  import json
2
3  def evaluate_retrieval(eval_dataset: list, retriever, k: int = 5):
4      hit_count = 0
5      rr_sum = 0.0
6
7      for item in eval_dataset:
8          results = retriever.retrieve(item["question"], top_k=k)
9          found_at = None
10         for rank, chunk in enumerate(results, start=1):
11             if item["relevant_passage"][:80] in chunk["text"]:
12                 found_at = rank
13                 break
14         if found_at:
15             hit_count += 1
16             rr_sum += 1.0 / found_at
17
18     hit_rate = hit_count / len(eval_dataset)
19     mrr = rr_sum / len(eval_dataset)
20     print(f"Hit Rate @{k}: {hit_rate:.3f}")
21     print(f"MRR: {mrr:.3f}")
22     return {"hit_rate": hit_rate, "mrr": mrr}
23
24 with open("eval_dataset.json") as f:
25     dataset = json.load(f)
26     evaluate_retrieval(dataset, retriever, k=5)

```

Listing 11: Minimal evaluation script

6.2 Qualitative Analysis

In your report, include a short qualitative analysis with:

- **2–3 success cases:** question asked, chunk retrieved, final generated answer.
- **1–2 failure cases:** what the system returned and why it was wrong (vocabulary mismatch, bad chunking boundary, LLM hallucination, etc.).

Important

The evaluation dataset and metric computation are **not compulsory** for a passing grade, but they are **strongly recommended**. Systems that include a thorough retrieval evaluation with qualitative analysis will receive a higher score. The goal is to build the professional habit of measuring what your systems actually do, not just what you expect them to do.

7 Technical Guidance

7.1 Calling the LLM from Flask

Both llama.cpp (llama-server) and vLLM expose an OpenAI-compatible `/v1/chat/completions` endpoint:

```

1  import requests, os
2
3  LLM_URL = os.getenv("LLM_URL", "http://llm:8080")
4
5  def call_llm(messages: list, max_tokens: int = 512) -> str:
6      payload = {"messages": messages, "max_tokens": max_tokens,
7                "temperature": 0.1, "stream": False}
8      resp = requests.post(f"{LLM_URL}/v1/chat/completions",
9                           json=payload, timeout=120)
10     resp.raise_for_status()
11     return resp.json()["choices"][0]["message"]["content"]
12
13 def build_rag_prompt(question: str, chunks: list) -> list:
14     context = "\n\n---\n\n".join(
15         [f"Source [{i+1}]: {c['text']}" for i, c in enumerate(chunks)]
16     )
17     return [
18         {
19             "role": "system",
20             "content": (
21                 "You are a helpful assistant. Answer based solely on the "
22                 "provided document excerpts. If the answer is not in the "
23                 "excerpts, say so clearly. Cite sources as [1], [2], etc."
24             )
25         },
26         {
27             "role": "user",
28             "content": f"Context:\n{context}\n\nQuestion: {question}"
29         }
30     ]

```

Listing 12: LLM call from Flask

7.2 ChromaDB as Vector Store

ChromaDB is the simplest vector store to deploy locally. Run it as a separate container and connect via HTTP:

```

1  services:
2    chromadb:
3      image: chromadb/chroma:latest
4      ports:
5        - "8001:8000"
6      volumes:
7        - chroma_data:/chroma/chroma
8
9  volumes:
10   chroma_data:

```

Listing 13: ChromaDB service in docker-compose.yml

```

1  import chromadb
2  from sentence_transformers import SentenceTransformer
3
4  chroma_client = chromadb.HttpClient(host="chromadb", port=8000)
5  collection    = chroma_client.get_or_create_collection("documents")
6  encoder       = SentenceTransformer("all-MiniLM-L6-v2")
7
8  def index_chunks(doc_id: str, filename: str, chunks: list):
9      embeddings = encoder.encode(chunks).tolist()
10     ids        = [f"{doc_id}_{i}" for i in range(len(chunks))]
11     metadatas  = [{"doc_id": doc_id, "filename": filename}
12                  for _ in chunks]
13     collection.add(documents=chunks, embeddings=embeddings,
14                   ids=ids, metadatas=metadatas)
15
16  def retrieve(query: str, top_k: int = 5) -> list:
17     query_emb = encoder.encode([query]).tolist()
18     results   = collection.query(
19         query_embeddings=query_emb, n_results=top_k,
20         include=["documents", "metadatas", "distances"]
21     )
22     return [
23         {"text": doc, "metadata": meta, "score": 1.0 - dist}
24         for doc, meta, dist in zip(
25             results["documents"][0],
26             results["metadatas"][0],
27             results["distances"][0]
28         )
29     ]

```

Listing 14: ChromaDB integration for indexing and retrieval

7.3 BM25 Retrieval Alternative

For a keyword-based approach that requires no vector database:

```

1  from rank_bm25 import BM25Okapi
2  import pickle
3
4  BM25_PATH = "/data/bm25_index.pkl"
5

```



```
6 def build_bm25_index(chunks: list):
7     tokenized = [c.lower().split() for c in chunks]
8     bm25 = BM25Okapi(tokenized)
9     with open(BM25_PATH, "wb") as f:
10         pickle.dump((bm25, chunks), f)
11
12 def retrieve_bm25(query: str, top_k: int = 5) -> list:
13     with open(BM25_PATH, "rb") as f:
14         bm25, chunks = pickle.load(f)
15     scores = bm25.get_scores(query.lower().split())
16     top_idx = sorted(range(len(scores)),
17                     key=lambda i: scores[i], reverse=True)[:top_k]
18     return [{"text": chunks[i], "score": float(scores[i])}
19             for i in top_idx]
```

Listing 15: BM25 retrieval with rank_bm25

7.4 Environment Variables and Service Discovery

Use environment variables in `docker-compose.yml` so containers can find each other by service name:

```
1 services:
2     flask_app:
3         build: ./flask_app
4         ports:
5             - "5000:5000"
6         environment:
7             - LLM_URL=http://llm:8080
8             - MINIO_URL=minio:9000
9             - MINIO_ACCESS_KEY=minioadmin
10            - MINIO_SECRET_KEY=minioadmin
11            - CHROMADB_URL=http://chromadb:8000
12        depends_on:
13            - llm
14            - minio
15            - chromadb
```

Listing 16: Flask environment and dependencies in docker-compose.yml

8 Deliverables and Submission

8.1 What to Submit

Submit a single link to a **public GitHub repository** via MiAulario by **March 11th, 2026**. The repository must contain:

1. All source code — Flask app, Dockerfiles, `docker-compose.yml`.
2. A `README.md` with clear setup and run instructions (see requirements below).
3. An evaluation dataset (`eval_dataset.json`) with at least 10 annotated question-answer pairs.

4. Evaluation results as a JSON or CSV file in **results/**.
5. A technical report (PDF, max 3 pages) in **report/**.

Important

Do not commit model weights to the repository. GGUF files are 4–8 GB. Provide a download script or clear manual instructions in your README specifying the exact filename and directory. The grader must be able to reproduce your setup within 10 minutes of cloning.

8.2 README Requirements

Your README.md must include:

1. **Architecture overview** — which LLM, retrieval method, and chunking strategy you chose.
2. **Model setup** — exact download URL and destination path for the model weights.
3. **Run instructions** — `docker-compose up` should start everything without further steps.
4. **API examples** — at least one `curl` snippet for document upload and one for query.
5. **Known limitations** — what is not implemented or known to fail.

8.3 Report Requirements (Max 3 Pages)

1. System Design (1 page)

- Architecture diagram (a clear box-and-arrow sketch is sufficient).
- Rationale for your LLM, chunking strategy, and retrieval method choices.
- Any significant engineering decisions or trade-offs.

2. Retrieval Evaluation (1 page)

- Description of the evaluation dataset: domain, document types, number of questions.
- Hit Rate @1, @3, @5 and MRR in a results table.
- 2–3 qualitative examples illustrating correct and incorrect system behavior.

3. Reflection (1 page)

- What worked well and what did not.
- One concrete improvement you would implement with more time.
- If you implemented the agentic extension: comparison against the simple RAG pipeline.

9 Evaluation Criteria

Component	Weight
Infrastructure: Dockerfiles + <code>docker-compose.yml</code> + one-command startup	20%
Document ingestion: MinIO storage + PDF/DOCX parsing + chunking	20%
Retrieval pipeline: at least one method, correctly integrated	20%
Flask API: all mandatory endpoints functional and well-documented	15%
LLM integration: correct prompt assembly, grounded generation	10%
Retrieval evaluation: dataset + metrics + qualitative analysis	10%
Report quality: clarity, design rationale, honest reflection	5%
Bonus: Frontend interface (containerized)	+5%
Bonus: Agentic pipeline (ReAct loop with multiple tools)	+5%

Table 6: Grading breakdown — base total: 100%, maximum with bonuses: 110%

Assessment philosophy: You are graded on *engineering quality*, not model performance. A system that starts cleanly with one command, has readable and documented code, and provides an honest evaluation of what it does well and poorly is the target. Correctness and reproducibility matter more than sophistication.

Common Pitfall

The most common failure modes that cost marks:

- Docker containers that do not start due to missing environment variables or absent model files.
- Hard-coded absolute paths that only work on your machine.
- No `/health` endpoint — the grader cannot verify the system is up.
- Retrieval returning empty results silently: always validate that your index is non-empty after ingestion.
- LLM generating confident answers when retrieved context is empty — handle this case explicitly.
- Missing `depends_on` in `docker-compose.yml` causing race conditions on startup.

10 Frequently Asked Questions

Can I use LangChain, LlamaIndex, or Haystack? Yes. You may use any orchestration library. However, you must still implement the Flask API layer yourself, and you must be able to explain every component in your report. Black-box pipelines with no understanding will not

receive full marks.

Can I use a different vector store? Yes. Qdrant, Weaviate, Milvus, Elasticsearch, or SQLite with `sqlite-vec` are all acceptable. Add the service to your `docker-compose.yml` and document how to run it.

My laptop has no GPU. Can I still complete the assignment? Yes. llama.cpp with a Q4_K_M GGUF model runs on CPU. Inference will be slower (15–30 seconds per query), but functionally complete. Use a 3B or 7B parameter model if memory is limited.

Can I use a cloud LLM API (OpenAI, Anthropic, Groq) instead of a local model? No. A locally deployed quantized model is a hard requirement. The learning objective is deploying an LLM, not calling an API. Systems using external API keys will not receive marks for the LLM integration component.

Can I use a different document store instead of MinIO? No. MinIO is a required component. You must store raw uploaded documents in MinIO and interact with it via the Python client.

How many documents should I use for testing? Five to ten documents is more than sufficient for development and evaluation. There is no minimum or maximum.

Can I implement multiple retrieval methods and compare them? Yes, and this is encouraged. Comparing BM25 to a bi-encoder on your evaluation dataset is exactly the kind of analysis that earns strong marks in the evaluation section.

11 Conclusion

This assignment synthesizes the practical skills from the course into a single deployable system. You are simultaneously a data engineer (parsing and chunking), a search engineer (retrieval pipeline), a backend developer (Flask REST API), and an ML engineer (LLM integration and evaluation). This is precisely the skill profile demanded of modern NLP engineers.

Two lessons are more important than any line of code:

- **Infrastructure determines success.** A correct retrieval method inside a broken deployment is worthless. Invest time early in making `docker-compose` up just work.
- **Measure before you conclude.** The retrieval evaluation, even on a ten-question dataset, will reveal failure modes invisible to manual testing. Build the habit.

Final Advice

Start with the infrastructure. Get all containers running and able to reach each other before writing any RAG logic. A `/health` endpoint confirming all four services are up is your first milestone.

Test each layer in isolation. Verify MinIO upload works, then verify parsing produces clean text on a real PDF, then verify retrieval returns non-empty results, then wire in the LLM. Debugging a fully integrated pipeline from scratch is much harder.

Model downloads are slow. A 4 GB GGUF file on a slow connection can take 30–60 minutes. Do not start the night before the deadline.

Commit early and often. Your Git history is visible and demonstrates steady progress. It also protects you if something breaks at the last minute.

Deadline: March 11th, 2026.

Good luck!